

PROJECT REPORT

Student Grade Management System

A Python console application to add, view, search, update and delete student records
with automatic grading and file storage

Submitted by	K Ravi Kumar
Roll Number	26BCE10135
Course / Class	B.Tech CSE
School	School of Computer science and engineering
Institute	Vellore Institute of Technology Bhopal
Faculty	Dr.sasmita padhy
Language	Python 3

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope
5. Users
6. Features
7. System Requirements
8. System Design
9. Implementation and Screenshots
10. Testing
11. Conclusion
12. Future Scope

1. Introduction

The Student Grade Management System is a simple console-based program written in Python. It helps a teacher or institute keep a record of students and their marks in three subjects: Python, Maths and English. The program calculates each student's average and grade automatically and stores all data in a text file so that nothing is lost when the program is closed.

2. Problem Statement

Marks are often written on paper or in scattered notes. This makes it slow to look up a student, calculate averages and grades, or find the top performer, and mistakes are easy to make. Data can also be lost when it is not stored properly. This project provides a small, reliable tool that manages student marks and saves them permanently.

3. Objectives

- Store student details (name, roll number and marks in three subjects).
- Perform add, view, search, update and delete operations.
- Calculate the average and grade of a student automatically.
- Find the student with the highest average.
- Save and load data from a file so records are permanent.
- Validate input so the program does not crash on wrong entries.

4. Scope

In scope

- Console (text menu) based application
- Three subjects: Python, Maths and English
- Automatic average and grade calculation
- Permanent storage in students.txt
- Add, view, search, update and delete operations
- Finding the highest scorer

Out of scope

- Graphical or web interface
- Multiple user accounts or passwords
- Database systems
- Attendance, fees or timetable management

5. Users

- **Teachers** enter marks and check student performance.
- **School or institute staff** look up student results quickly.
- **Students** (through a teacher) check their average and grade.

6. Features

No.	Feature	Description
1	Add Student	Stores name, roll number and marks. Rejects duplicate roll numbers and invalid marks.
2	View Students	Lists every student record.
3	Search Student	Finds a student by roll number and shows average and grade.
4	Find Highest Scorer	Shows the student with the best average.
5	Update Student	Changes the marks of an existing student.
6	Delete Student	Removes a student record.
7	File Storage	Loads records at start and saves after every change.

Grading Rules

Average	Grade
90 and above	A+
80 to 89	A
70 to 79	B
60 to 69	C
50 to 59	D
Below 50	F

7. System Requirements

- Operating system: Windows, Linux or macOS
- Software: Python 3.6 or newer
- Libraries: none (only the Python standard library: csv and os)
- Hardware: any computer that can run Python

8. System Design

The project is split into small files so each part has one job.

File	Purpose
main.py	Shows the menu and calls the correct function for the user's choice.
student.py	Contains the add, view, search, update, delete and highest scorer operations, plus average and grade calculation.

File	Purpose
file_handler.py	Loads students from students.txt and saves them back.
students.txt	Stores the records, one student per line.

Data format. Each line of students.txt holds one student in CSV form:

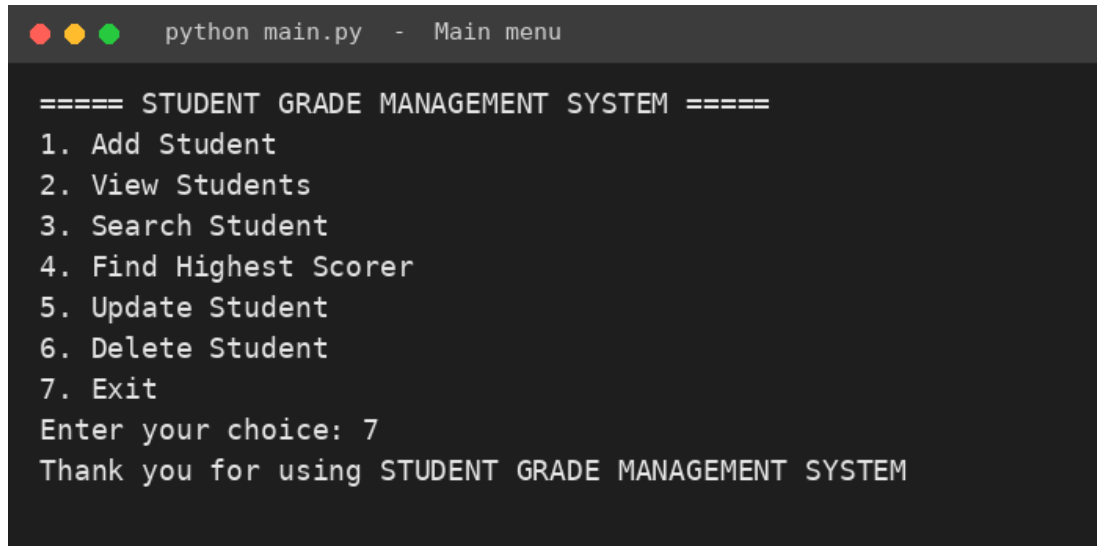
```
name,roll,python,maths,english
Aarav Sharma,101,85.0,90.0,78.0
```

Working. When the program starts, main.py calls load_students() to read the file into a list of dictionaries. Every add, update or delete changes that list and then calls save_students() to write it back to the file. Because the file is rewritten after each change, the data stays safe even if the program is closed suddenly.

9. Implementation and Screenshots

The following screenshots show the program running. Each one was produced by running main.py with the inputs shown.

9.1 Main Menu

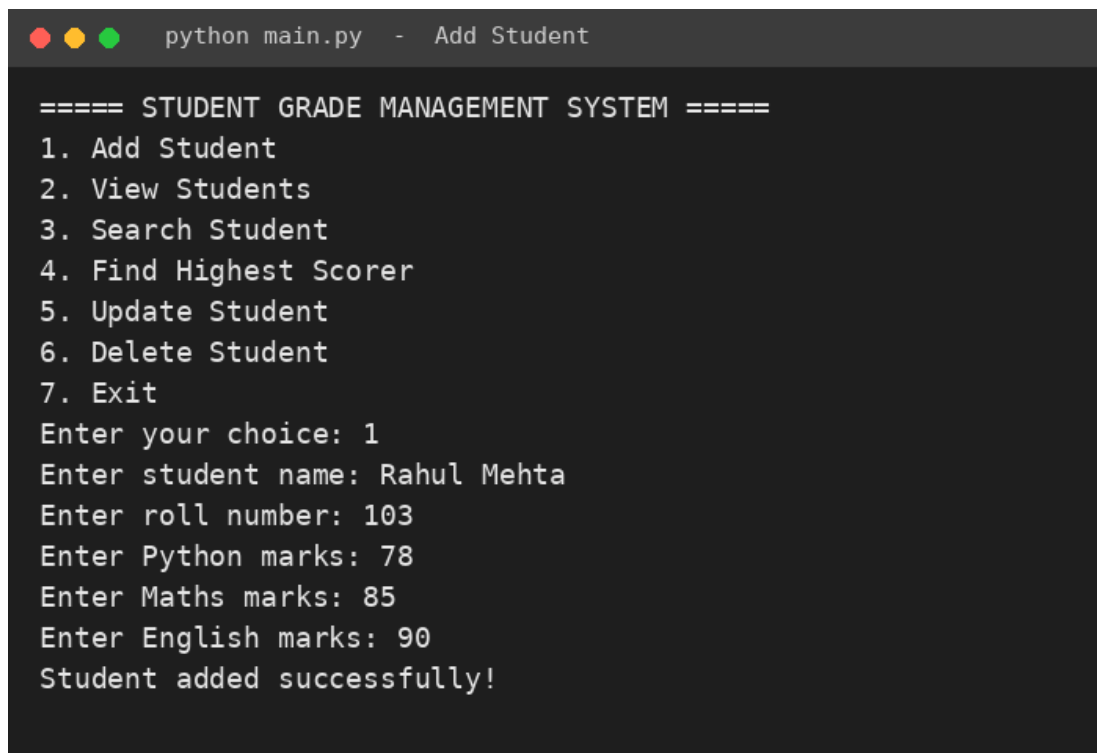
A screenshot of a terminal window titled 'python main.py - Main menu'. The terminal displays a menu for the 'STUDENT GRADE MANAGEMENT SYSTEM' with seven options: 1. Add Student, 2. View Students, 3. Search Student, 4. Find Highest Scorer, 5. Update Student, 6. Delete Student, and 7. Exit. Below the menu, the user has entered '7' as their choice, and the program has responded with 'Thank you for using STUDENT GRADE MANAGEMENT SYSTEM'.

```
python main.py - Main menu

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 7
Thank you for using STUDENT GRADE MANAGEMENT SYSTEM
```

Figure 1: Main menu of the program

9.2 Add Student

A screenshot of a terminal window titled 'python main.py - Add Student'. The terminal shows the same menu as in Figure 1. The user has entered '1' to select 'Add Student'. The program then prompts for student details: 'Enter student name: Rahul Mehta', 'Enter roll number: 103', 'Enter Python marks: 78', 'Enter Maths marks: 85', and 'Enter English marks: 90'. After receiving all inputs, the program outputs 'Student added successfully!'.

```
python main.py - Add Student

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 1
Enter student name: Rahul Mehta
Enter roll number: 103
Enter Python marks: 78
Enter Maths marks: 85
Enter English marks: 90
Student added successfully!
```

Figure 2: Adding a new student

9.3 View Students

```
python main.py - View Students

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 2

===== STUDENT DETAILS =====
Name: Aarav Sharma
Roll Number: 101
Python: 85.0
Maths: 90.0
English: 78.0
-----
Name: Priya Verma
Roll Number: 102
Python: 92.0
Maths: 88.0
English: 95.0
-----
Name: Rahul Mehta
Roll Number: 103
Python: 78.0
Maths: 85.0
English: 90.0
-----
```

Figure 3: Viewing all student records

9.4 Search Student

```
python main.py - Search Student

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 3
Enter roll number to search: 103

Student Found!
Name: Rahul Mehta
Roll Number: 103
Python: 78.0
Maths: 85.0
English: 90.0
Average: 84.33
Grade: A
```

Figure 4: Searching by roll number shows average and grade

9.5 Highest Scorer



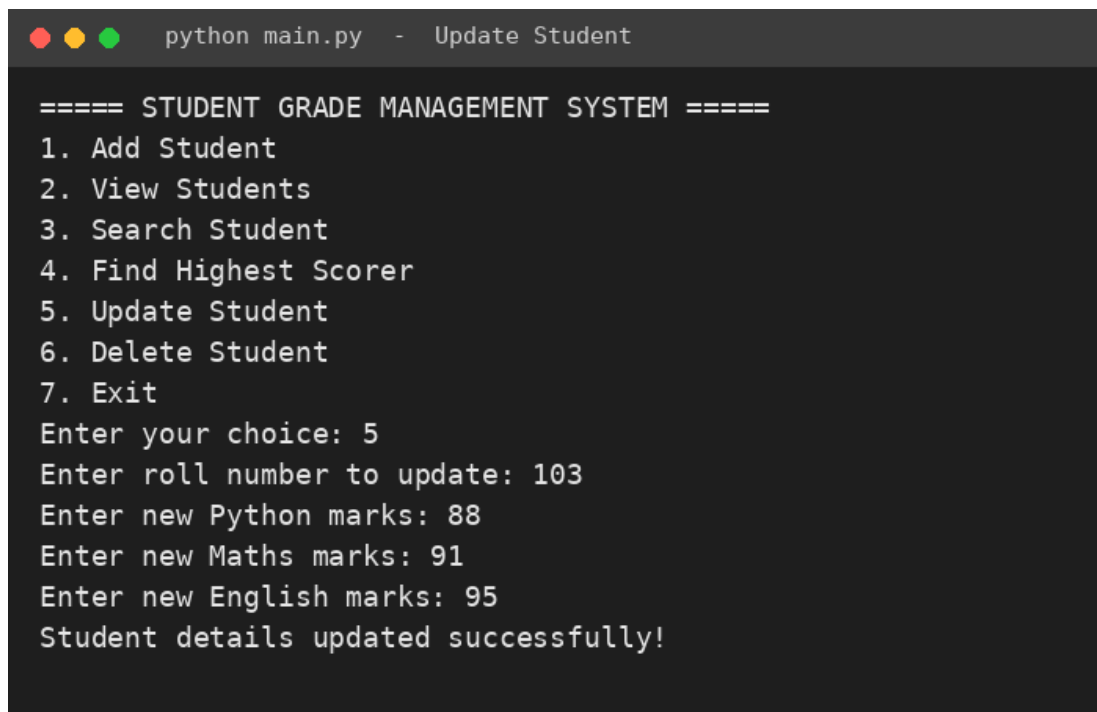
```
python main.py - Find Highest Scorer

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 4

===== HIGHEST SCORER =====
Name: Priya Verma
Roll Number: 102
Average: 91.67
Grade: A+
```

Figure 5: Student with the highest average

9.6 Update Student

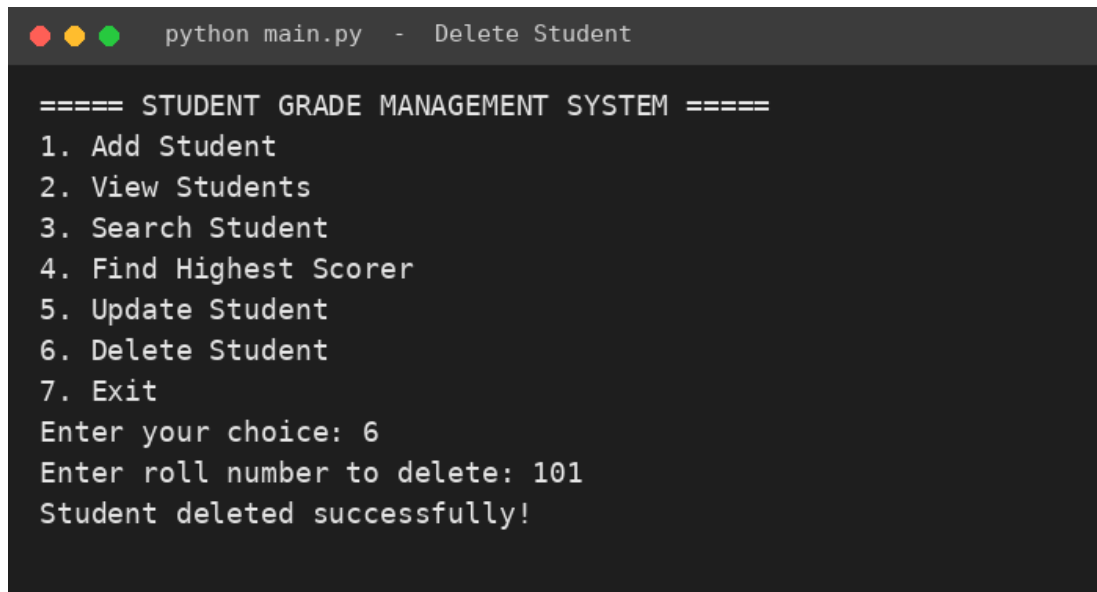


```
python main.py - Update Student

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 5
Enter roll number to update: 103
Enter new Python marks: 88
Enter new Maths marks: 91
Enter new English marks: 95
Student details updated successfully!
```

Figure 6: Updating a student's marks

9.7 Delete Student

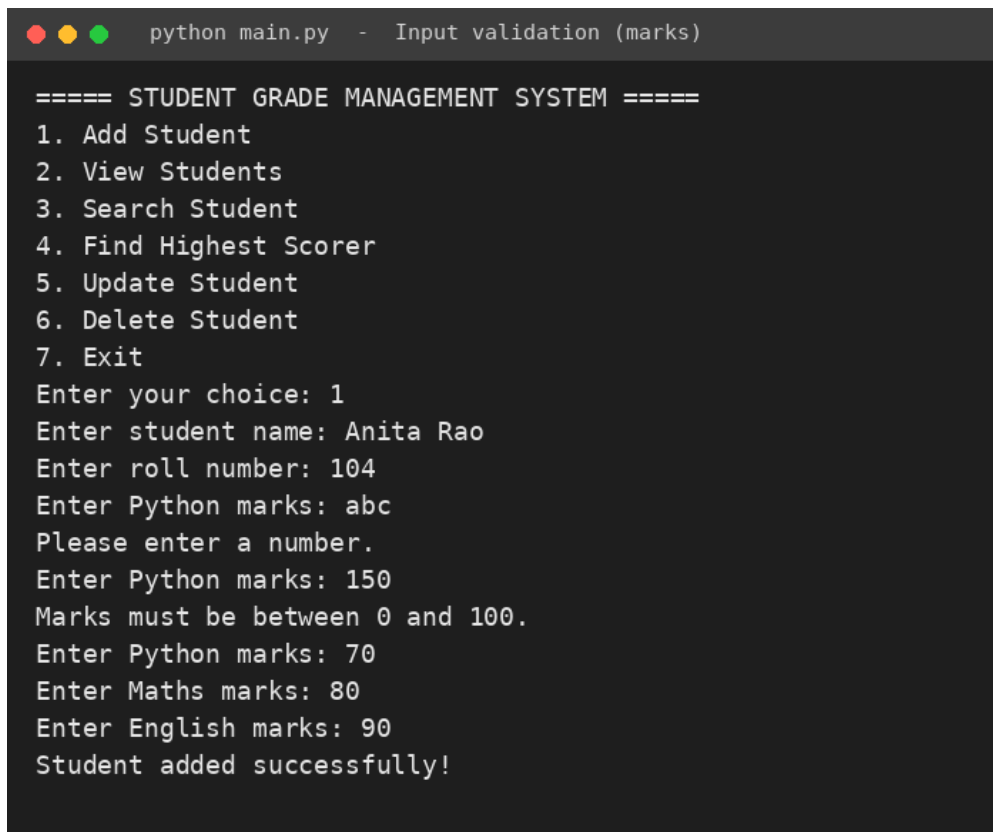


```
python main.py - Delete Student

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 6
Enter roll number to delete: 101
Student deleted successfully!
```

Figure 7: Deleting a student record

9.8 Input Validation



```
python main.py - Input validation (marks)

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 1
Enter student name: Anita Rao
Enter roll number: 104
Enter Python marks: abc
Please enter a number.
Enter Python marks: 150
Marks must be between 0 and 100.
Enter Python marks: 70
Enter Maths marks: 80
Enter English marks: 90
Student added successfully!
```

Figure 8: Text and out-of-range marks are rejected

9.9 Error Handling

```
python main.py - Duplicate roll and not found

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 1
Enter student name: Test
Enter roll number: 102
A student with this roll number already exists.

===== STUDENT GRADE MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Find Highest Scorer
5. Update Student
6. Delete Student
7. Exit
Enter your choice: 3
Enter roll number to search: 999
Student not found.
```

Figure 9: Duplicate roll number and student not found

10. Testing

Test	Input	Expected result	Result
Add valid student	Rahul Mehta, 103, 78, 85, 90	Student is added and saved	Pass
Duplicate roll number	Roll 102 again	Rejected with a message	Pass
Marks as text	abc	Asks again, no crash	Pass
Marks above 100	150	Asks again	Pass
Search existing	Roll 103	Shows details, average and grade	Pass
Search missing	Roll 999	Student not found	Pass
Highest scorer	Option 4	Shows top average	Pass
Update marks	Roll 103, new marks	Marks change and are saved	Pass
Delete student	Roll 101	Record removed and saved	Pass
Invalid menu choice	9	Invalid choice message	Pass

11. Conclusion

The Student Grade Management System meets its objectives. It manages student records with add, view, search, update and delete operations, calculates averages and grades correctly, and stores data permanently in a file. Splitting the code into main.py, student.py and file_handler.py keeps it easy to read and maintain, and input checks stop common mistakes from crashing the program.

12. Future Scope

- Support more subjects.
- Sort students by marks or name.
- Export a printable report card for each student.
- Add a graphical interface or a web version.
- Use a database such as SQLite for larger data.